

Programming for Data Science: Lab #10

April 13, 2026

Professor Carlos J. Costa

Cheng Amy QIAN

Problem 1

Explain the syntax used in each framework.

Solution

1. TensorFlow / Keras: Declarative and Highly Abstracted

TensorFlow, utilizing its Keras API, relies on a declarative syntax. The developer specifies *what* the model architecture should be, and the framework automatically handles the underlying computational execution graphs.

```
1 model_tf = keras.Sequential([
2     layers.Dense(64, activation="relu", input_shape=(input_dim,)),
3     layers.Dense(32, activation="relu"),
4     layers.Dense(1, activation="sigmoid"),
5 ])
6
7 model_tf.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
8 model_tf.fit(X_train, y_train, epochs=5, batch_size=32)
```

Listing 1: Keras Sequential example

Key Characteristics:

- **Model Definition:** The `Sequential` API allows for highly readable, Lego-like stacking of layers. Activation functions are conveniently passed as string arguments directly within the layer instantiation.
- **Compilation Phase:** Before training, the `.compile()` method is required. This abstracts the assignment of the optimizer, the loss function, and the evaluation metrics into a single, clean line.
- **Abstracted Training Loop:** The `.fit()` method is a massive abstraction. It automatically handles batching the data, performing forward passes, calculating loss, executing backpropagation, and updating the weights without exposing these mechanics to the user.

2. PyTorch: Imperative and Object-Oriented

PyTorch employs an imperative, "define-by-run" syntax. It requires a highly explicit, object-oriented approach where the developer dictates exactly *how* the tensors flow through the network mathematically.

```
1 class MLP(nn.Module):
2     def __init__(self, input_dim):
3         super().__init__()
4         self.net = nn.Sequential(
5             nn.Linear(input_dim, 64),
6             nn.ReLU(),
7             nn.Linear(64, 32),
8             nn.ReLU(),
9             nn.Linear(32, 1),
10            nn.Sigmoid(),
11        )
12
13    def forward(self, x):
14        return self.net(x)
15
```

```

16 # --- Explicit Training Loop ---
17 for epoch in range(epochs):
18     model_torch.train()           # 1. Set to training mode
19     outputs = model_torch(X_train_t) # 2. Forward pass
20     loss = criterion(outputs, y_train_t) # 3. Calculate loss
21     optimizer.zero_grad()          # 4. Clear previous gradients
22     loss.backward()                 # 5. Backpropagation
23     optimizer.step()                # 6. Update weights

```

Listing 2: PyTorch explicit training loop

Key Characteristics:

- **Model Definition:** Models are strictly defined as Python classes inheriting from `nn.Module`. The layers (and their standalone activation functions) are initialized in the `__init__` constructor. The forward pass—how the inputs traverse the layers—is explicitly coded in the `forward()` method.
- **Manual Training Loop:** Unlike Keras, PyTorch requires the user to construct the training loop manually. The developer is responsible for tracking gradients, stepping the optimizer, and toggling the model between `.train()` and `.eval()` modes (which changes the behavior of layers like Dropout or BatchNorm). While this requires more boilerplate code, it grants researchers maximum flexibility for custom training dynamics.

Overall, would opt for **Keras** when the project is standard and when the speed of deployment is critical. Syntax is easily readable for beginners or cross-functional teams. But would opt for **PyTorch** when doing research, building custom loss functions, or implementing complex non-sequential architectures.

Problem 2

Describe how the model architecture is defined.

Solution

While the syntax differs, both frameworks rely on assembling foundational layers to define the architecture.

TensorFlow / Keras: The architecture is built by passing a list of layer objects to a `Sequential` container.

- **Input Specification:** The first layer strictly requires an `input_shape` or `input_dim` argument to initialize the weight matrices.
- **Layer Diversity:** As seen in `CompareArchitectures.ipynb`, Keras easily swaps architectures. A standard MLP uses `layers.Dense`, an RNN uses `layers.Embedding` followed by `layers.LSTM`, and a CNN uses `layers.Conv1D` combined with `layers.GlobalMaxPooling1D`.
- **Activations:** Activation functions (e.g., `'relu'`, `'sigmoid'`) are typically defined as arguments directly inside the layer definition for brevity.

PyTorch: The architecture is defined explicitly as a computational graph inside a class.

- **Initialization:** In the `__init__` method, layers like `nn.Linear` (equivalent to Keras's `Dense`) are instantiated. PyTorch requires specifying both the `in_features` and `out_features` for each linear layer, forcing the developer to track matrix dimensions manually.
- **Activations as Layers:** Unlike Keras, activation functions in PyTorch are often instantiated as distinct layers (e.g., `nn.ReLU()`, `nn.Sigmoid()`) within the `nn.Sequential` block or applied functionally during the `forward` pass.

Problem 3

Specify what data preparation steps are required.

Solution

Neural networks cannot process raw text; therefore, rigorous data preparation is required before training.

Universal Steps (Framework Agnostic):

- **Data Cleaning:** Dropping missing values using Pandas (`df.dropna()`).
- **Train/Test Split:** Segregating data into training and testing sets to evaluate generalization (`train_test_split`).
- **Text Vectorization:** Converting text to numerical formats. The MLP models use TF-IDF (`TfidfVectorizer`), which calculates word frequency distributions. The LSTM and CNN models use Tokenization (`Tokenizer`) combined with `pad_sequences` to ensure all input sequences have a uniform length (e.g., `maxlen=100`), preserving the sequential order of words.

Framework-Specific Preparations:

- **TensorFlow/Keras:** Keras is highly flexible and natively accepts standard NumPy arrays or padded sequences output directly from the vectorization steps. No further data type casting is strictly required.
- **PyTorch:** PyTorch is strictly typed and relies on its own Tensor structures. The NumPy arrays must be explicitly converted using `torch.tensor(..., dtype=torch.float32)`. Furthermore, PyTorch expects the target labels (y) to match the output shape of the network perfectly. This requires reshaping the 1D label array into a 2D column vector using `.view(-1, 1)`.

Problem 4

Explain how models are trained.

Solution

The training phase further emphasizes the architectural philosophies of both frameworks. TensorFlow/Keras completely abstracts the training loop, while PyTorch forces the developer to manually orchestrate every step of the backpropagation algorithm.

TensorFlow / Keras: Automated Training

Keras splits the training process into two distinct, highly abstracted method calls:

- **Compilation (`.compile()`):** Before training, the model must be compiled. This step attaches the loss function (e.g., `'binary_crossentropy'` for binary classification), the optimizer (e.g., `'adam'`), and the evaluation metrics (e.g., `['accuracy']`) to the computational graph.
- **Fitting (`.fit()`):** The actual training loop is invoked with a single command. By passing X_{train} , y_{train} , `epochs=5`, and `batch_size=32`, Keras automatically handles the slicing of data into mini-batches, computes the forward pass, calculates the loss, performs backpropagation, and updates the weights for every batch across every epoch.

PyTorch: Manual Optimization Loop

PyTorch requires explicit instantiation of the loss and optimization functions, followed by a manually constructed `for` loop to iterate over the epochs.

- **Setup:** The loss function is instantiated as an object (`criterion = nn.BCELoss()`), and the optimizer is linked directly to the model's parameters (`optimizer = optim.Adam(model_torch.parameters(), lr=0.001)`).
- **The Training Loop:** For each epoch, the developer must explicitly define the sequence of operations:
 1. `model_torch.train()`: Sets the model to training mode (enabling behaviors like Dropout).
 2. `outputs = model_torch(X_train_t)`: Executes the forward pass to get predictions.
 3. `loss = criterion(outputs, y_train_t)`: Computes the error between predictions and true labels.
 4. `optimizer.zero_grad()`: Clears the gradients from the previous iteration. This is critical, as PyTorch accumulates gradients by default.
 5. `loss.backward()`: Triggers backpropagation, computing the derivative of the loss with respect to every parameter.
 6. `optimizer.step()`: Updates the model's weights based on the computed gradients.

A crucial difference observed in the `CompareFrameworks.ipynb` notebook is that the Keras model utilizes mini-batch stochastic gradient descent (`batch_size=32`), whereas the PyTorch implementation passes the entire dataset tensor at once, effectively performing full-batch gradient descent.

Problem 5

Why do results differ when using the same model in different frameworks?

Solution

When executing the theoretically identical Multi-Layer Perceptron (MLP) architecture in both TensorFlow/Keras and PyTorch, the final accuracy and loss metrics diverge. This discrepancy is not due to a flaw in the underlying mathematics of either framework, but rather distinct differences in implementation details, default behaviors, and the specific training strategies coded in the `CompareFrameworks.ipynb` notebook.

1. Discrepancy in Batching Strategy (The Primary Cause)

The most significant reason for the differing results lies in how the training data is fed into the models within the provided code:

- **Keras (Mini-Batch Gradient Descent):** The Keras implementation explicitly uses `batch_size=32` during `.fit()`. This means the model updates its weights multiple times per epoch (specifically, $N/32$ times, where N is the size of the training set). Mini-batching introduces stochastic noise into the gradient estimation, which often helps the model escape local minima and generalize better.
- **PyTorch (Full-Batch Gradient Descent):** The PyTorch training loop lacks a `DataLoader`. Instead, it passes the entire training tensor `X_train_t` through the model simultaneously. Consequently, the optimizer performs only exactly *one* weight update per epoch based on the average gradient of the entire dataset.

2. Number of Optimization Steps

Because of the batching discrepancy, the total number of weight updates (optimization steps) differs drastically:

- The Keras model runs for 5 epochs. If the training set has, for example, 3,200 samples, there are 100 batches per epoch. Over 5 epochs, the model undergoes $100 \times 5 = 500$ weight updates.
- The PyTorch model runs for 20 epochs but updates only once per epoch, resulting in a mere 20 total weight updates. The Keras model reaches a much more refined region of the loss landscape simply because it has taken significantly more steps.

3. Default Weight Initialization Schemes

Even if the training loops were perfectly identical, initial results would still differ due to how each framework randomly initializes the starting weights of the layers:

- **TensorFlow/Keras** typically defaults to *Glorot Uniform* (also known as Xavier initialization) for Dense layers.
- **PyTorch** usually defaults to a variation of *Kaiming Uniform* (He initialization), which is scaled by the inverse square root of the layer's input features.
- Because the starting positions in the loss landscape are different, gradient descent will carve slightly different paths to the final minimum.

4. Optimizer Epsilon and Hyperparameters

While both models use the Adam optimizer, the default smoothing term (ϵ) designed to prevent division by zero differs slightly between the frameworks (often $1e-7$ in Keras vs. $1e-8$ in PyTorch). Over many epochs, these microscopic differences can compound, leading to different final weights.

Problem 6

Why do results differ when using different architectures with the same framework?

Solution

In the `CompareArchitectures.ipynb` notebook, we evaluate three distinct neural network architectures (MLP, LSTM, and CNN) built entirely within TensorFlow/Keras. Because the framework and training loops are identical, the differences in performance stem entirely from how each architecture ingests, represents, and processes textual data.

1. Multi-Layer Perceptron (MLP): The "Bag of Words" Approach

- **Data Representation:** The MLP uses a `TfidfVectorizer`. This converts a document into a flat vector of word frequencies (scaled by their inverse document frequency), capped at 5,000 features.
- **Processing Mechanism:** MLPs treat the input as an unordered "bag of words." The model looks exclusively at which words are present and how often they appear, passing this through dense layers to find non-linear correlations with the labels.

- **Why Results Differ:** The MLP fundamentally destroys sentence structure. It cannot understand syntax, context, or the sequential relationships between words (e.g., it cannot distinguish between "the news was not fake" and "the fake was not news"). While effective for simple keyword-based classification, it struggles with complex linguistic nuances.

2. Long Short-Term Memory (LSTM): The Contextual Sequence Approach

- **Data Representation:** The LSTM uses a `Tokenizer` and `pad_sequences` to convert text into an ordered array of integer tokens, followed by an `Embedding` layer that maps each word to a dense 64-dimensional semantic vector.
- **Processing Mechanism:** As a Recurrent Neural Network (RNN), the LSTM processes the sentence sequentially, word by word. It maintains an internal "hidden state" and "cell state" (memory) that updates as it reads the sequence.
- **Why Results Differ:** LSTMs are explicitly designed to capture long-term dependencies and context in sequential data. They understand the order of words, making them highly effective at capturing the semantic meaning, tone, and logical flow of a sentence—features that are highly indicative of deceptive or "fake" news.

3. Convolutional Neural Network (CNN): The Local Pattern Approach

- **Data Representation:** Like the LSTM, the CNN uses tokenized sequences and an `Embedding` layer to preserve word order and semantic richness.
- **Processing Mechanism:** Instead of reading sequentially, the CNN uses a 1D Convolutional layer (`Conv1D`) with a kernel size of 5. This acts as a sliding window that scans across the sentence, looking at 5 words at a time. It is then followed by a `GlobalMaxPooling1D` layer, which extracts the strongest signals from the entire text.
- **Why Results Differ:** The CNN acts as an advanced n-gram detector. It is incredibly efficient at identifying specific, localized linguistic patterns or "trigger phrases" (e.g., sensationalist groupings of words like "shocking new discovery") regardless of where they appear in the text.

Results differ because each architecture optimizes for a different aspect of language. The MLP optimizes for keyword frequency, the LSTM optimizes for sequential context, and the CNN optimizes for local structural patterns. In NLP tasks like fake news detection, sequence-based models (LSTM/CNN) generally outperform flat models (MLP) because word order and context carry the bulk of the semantic meaning.

Problem 7

Possible improvements: Identify improvements in the code, considering data preprocessing, model design, training strategy, evaluation, and code structure.

Solution

Implementing the following improvements would enhance model performance, robustness, and interpretability.

1. Data Preprocessing

- **Text Normalization:** Currently, the text is only filtered for missing values (`df.dropna()`). The code must be updated to clean the raw text by converting it to lowercase, removing punctuation, stripping URLs/HTML tags, and removing standard English stop words (e.g., "the", "is", "at").
- **Lemmatization/Stemming:** Reducing words to their base roots (e.g., "running" → "run") reduces the vocabulary size and helps the models group similar semantic concepts together.
- **Stratified Splitting:** In `CompareArchitectures.ipynb`, the `train_test_split` does not use the `stratify=y` argument. If the fake news dataset is imbalanced, this could result in an uneven distribution of target labels in the test set.

2. Model Design

- **Regularization:** None of the architectures employ techniques to prevent overfitting. Adding `Dropout` layers (e.g., `layers.Dropout(0.5)`) after the Dense or LSTM layers is crucial to force the network to learn robust features rather than memorizing the training data.
- **Pre-trained Word Embeddings:** The sequence models use randomly initialized `Embedding` layers. Leveraging pre-trained embeddings (such as GloVe, Word2Vec, or FastText) would transfer vast amounts of prior linguistic knowledge into the model, significantly boosting the LSTM and CNN performance.

3. Training Strategy

- **Validation Set Integration:** The Keras `.fit()` methods do not include a validation split (e.g., `validation_split=0.2`). Without monitoring a validation set during training, it is impossible to know at which epoch the model begins to overfit.
- **Early Stopping:** The epochs are hardcoded (5 or 20). Implementing an `EarlyStopping` callback would automatically halt training when the validation loss stops improving, saving computational resources and preventing overtraining.
- **PyTorch Mini-batching:** As discussed previously, the PyTorch implementation performs full-batch gradient descent. It urgently needs a `torch.utils.data.DataLoader` to implement mini-batching, which would stabilize the optimization path and align its training dynamics with the Keras models.

4. Evaluation Metrics

- **Beyond Accuracy:** The models are evaluated solely using Accuracy. In binary classification (especially fake news detection, where classes might be imbalanced), accuracy is heavily misleading.
- **Comprehensive Reporting:** The evaluation code should be expanded to generate a Confusion Matrix and calculate Precision, Recall, and the F1-Score. For instance, in fake news detection, minimizing False Negatives (missing a fake article) or False Positives (flagging real news as fake) often requires optimizing for Recall or Precision over raw Accuracy.

5. Code Structure

- **Hyperparameter Centralization:** Magic numbers (like `max_features=5000`, `maxlen=100`, `batch_size=32`) are scattered throughout the code. These should be extracted into a centralized configuration dictionary at the top of the notebook for easier experimentation.

- **Modularization:** The evaluation printing logic is repeated for every model. This violates the DRY (Don't Repeat Yourself) principle and should be encapsulated into a reusable Python function.

Demonstration of Improvements (Keras LSTM)

Below is a refactored implementation of the LSTM model demonstrating the suggested improvements in code structure, preprocessing, regularization, training strategy, and evaluation metrics:

```

1 import pandas as pd
2 import re
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import classification_report, confusion_matrix
5 from tensorflow.keras.preprocessing.text import Tokenizer
6 from tensorflow.keras.preprocessing.sequence import pad_sequences
7 from tensorflow import keras
8 from tensorflow.keras import layers
9 from tensorflow.keras.callbacks import EarlyStopping
10
11 # 1. CODE STRUCTURE: Centralized Hyperparameters
12 CONFIG = {
13     "max_features": 5000,
14     "maxlen": 100,
15     "embedding_dim": 64,
16     "batch_size": 32,
17     "epochs": 20, # Increased, but safely bounded by Early Stopping
18     "test_size": 0.2,
19     "random_state": 42,
20 }
21
22 # 2. DATA PREPROCESSING: Text Cleaning Function
23 def clean_text(text):
24     text = str(text).lower() # Lowercase
25     text = re.sub(r"<[^>+>", "", text) # Remove HTML tags
26     text = re.sub(r"[^\w\s]", "", text) # Remove punctuation
27     return text
28
29 # Load and clean data
30 url = "https://github.com/masterfloss/data/raw/refs/heads/main/fake_new.xlsx"
31 df = pd.read_excel(url).dropna()
32 df["clean_text"] = df["text"].apply(clean_text)
33
34 # 3. DATA PREPROCESSING: Stratified Splitting
35 X_train_text, X_test_text, y_train, y_test = train_test_split(
36     df["clean_text"],
37     df["label"],
38     test_size=CONFIG["test_size"],
39     random_state=CONFIG["random_state"],
40     stratify=df["label"], # Prevents train/test class imbalance
41 )
42
43 # Tokenization and Padding
44 tokenizer = Tokenizer(num_words=CONFIG["max_features"])
45 tokenizer.fit_on_texts(X_train_text)
46 X_train_pad = pad_sequences(tokenizer.texts_to_sequences(X_train_text),

```

```

47         maxlen=CONFIG["maxlen"])
48 X_test_pad = pad_sequences(tokenizer.texts_to_sequences(X_test_text),
49                             maxlen=CONFIG["maxlen"])
50
51 # 4. MODEL DESIGN: Regularization via Dropout
52 improved_lstm = keras.Sequential([
53     layers.Embedding(CONFIG["max_features"], CONFIG["embedding_dim"],
54                     input_length=CONFIG["maxlen"]),
55     layers.LSTM(64, return_sequences=False),
56     layers.Dropout(0.5),                # Prevents overfitting
57     layers.Dense(32, activation="relu"),
58     layers.Dropout(0.5),                # Prevents overfitting
59     layers.Dense(1, activation="sigmoid"),
60 ])
61
62 improved_lstm.compile(optimizer="adam", loss="binary_crossentropy",
63                       metrics=["accuracy"])
64
65 # 5. TRAINING STRATEGY: Early Stopping and Validation Split
66 early_stopping = EarlyStopping(
67     monitor="val_loss",
68     patience=3,                        # Stops if no improvement after 3 epochs
69     restore_best_weights=True,        # Rolls back to the best model state
70 )
71
72 history = improved_lstm.fit(
73     X_train_pad, y_train,
74     epochs=CONFIG["epochs"],
75     batch_size=CONFIG["batch_size"],
76     validation_split=0.2,              # Monitors unseen data during training
77     callbacks=[early_stopping],       # Attaches the early stopping mechanism
78     verbose=1,
79 )
80
81 # 6. EVALUATION: Modular Function & Comprehensive Metrics
82 def evaluate_model(model, X_test, y_test, model_name="Model"):
83     print(f"--- Evaluation: {model_name} ---")
84     y_pred_probs = model.predict(X_test)
85     y_pred = (y_pred_probs > 0.5).astype(int)
86
87     print("\nConfusion Matrix:")
88     print(confusion_matrix(y_test, y_pred))
89
90     print("\nClassification Report (Precision, Recall, F1-Score):")
91     print(classification_report(y_test, y_pred))
92
93 # Execute evaluation
94 evaluate_model(improved_lstm, X_test_pad, y_test, "Improved LSTM")

```

Listing 3: Refactored Keras LSTM with cleaning and early stopping

Added `CONFIG` for clean code architecture. Added `Regex`, `re`, for text manipulation. Added `Scikitlearn` metrics, `classification_report`, which automatically calculate the precision, recall, and F1 scores. Added `Keras Callbacks`, `EarlyStopping`, to automate model monitoring.